Events & Buttons

Learning objectives

- Explain how to add an event listener for a button click
- Build a component with a button that has a click event

Recap

We've seen previously how we handle events and inputs in JS

Q. What's happening in this code snippet?

```
<!-- HTML -->
<button>Click Me!</button>

// JS
const button = document.querySelector("button");

button.addEventListener("click", handlerFunc);

function handlerFunc(event) {
 //Do stuff
}
```

Recap

React gives us the same ability of dealing with events, but with its friendly twist!

Let's have a closer look at what's happening here

```
// JSX
<button onClick={handlerFunc}>
  Click Me!
</button>;

// JS
function handlerFunc(event) {
 // Do Stuff
}
```

Recap

First, the event listener is **added as an attribute** to the HTML element you're targeting.

This way, through JSX, you **combine** both **querying** and **attaching** the listener to an element!

```
// JSX
<button onClick={handlerFunc}>
  Click Me!
</button>;

// JS
function handlerFunc(event) {
 // Do Stuff
}
```

Recap

The **name** of the attribute determines which **event** you are targeting, and it should always be **camelCase**.

You can use **any** event listed in the MDN reference, prefixed with "**on**"

```
// JSX
<button onClick={handlerFunc}>
  Click Me!
</button>;

// JS
function handlerFunc(event) {
 // Do Stuff
}
```

Recap

Just like with plain JS, we need a **handler** function that will be **called** when the event triggers

Notice that, as always, you have an **event object** that is passed to the handler function

```
// JSX
<button onClick={handlerFunc}>
  Click Me!
</button>;

// JS
function handlerFunc(event) {
 // Do Stuff
}
```

Recap

We still need to consider the **default behaviour** of elements, and use **preventDefault()** when needed

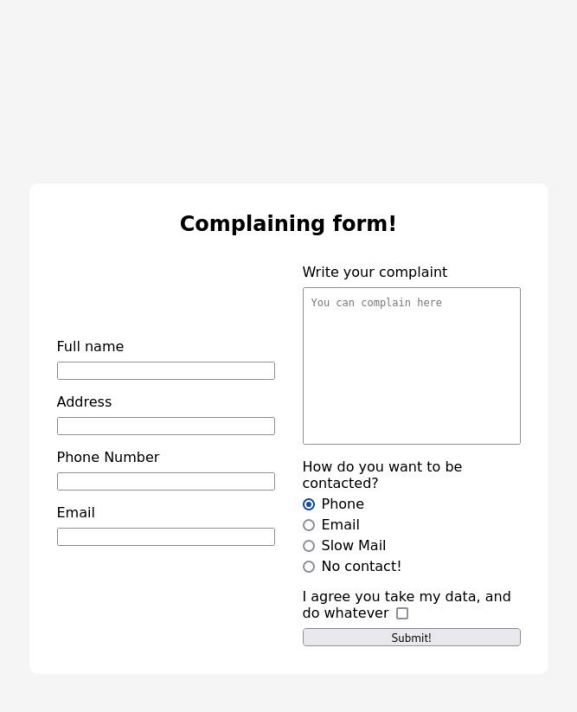
```
// JSX

<input type="submit" onSubmit={handleSubmit}
value="Submit"/>

// JS

function handleSubmit(event) {
 event.preventDefault()
 // Do Stuff
}
```

Events & Forms

Learning objectives

- Use event listeners for forms, including submit and change events
- Explain the difference between controlled and uncontrolled elements, including form elements.
- Build a component with a controlled form that has multiple inputs of different types
- Refactor a component that uses multiple state hooks to use one that dynamically handles multiple inputs

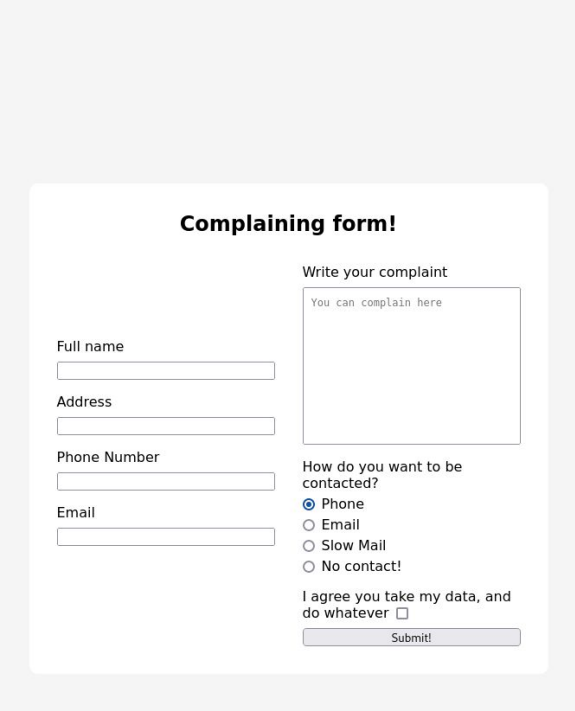
Uncontrolled forms



We've seen before how we interacted with forms using Vanilla JS

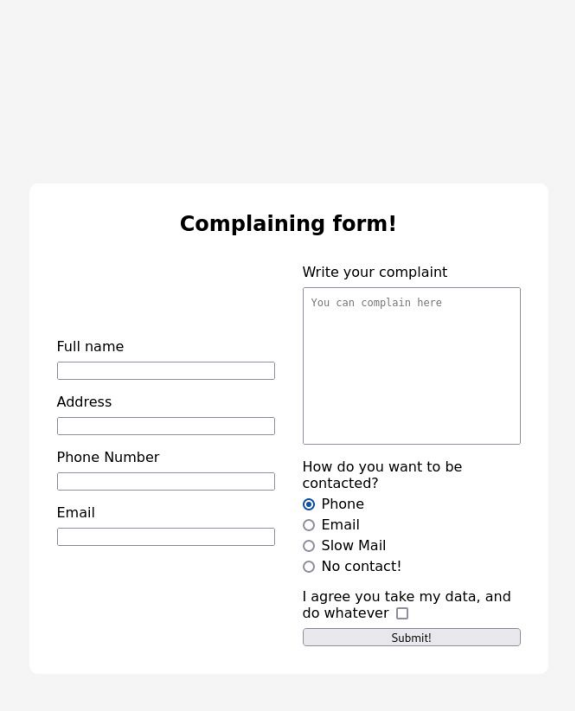
Q. How would you submit this form using JS?

Uncontrolled forms



Check it out here

Uncontrolled forms



Q. And now with React?

Uncontrolled forms

```
//JSX
<form className="form" onSubmit={submitForm}>
 <h2>Complaining form!</h2>
 <div className="form__section-left">
   <label>
     Full name
     <input
       type="text"
       name="name"
     />
   </label>
     // Rest of form inputs...
   </div>
 <input type="submit" value="Submit!" />
</form>;
```

We can just use the **same** process as **before**, just with some React sprinkles!

Uncontrolled forms

```
//JSX
<form className="form" onSubmit={handleSubmit}>
 <h2>Complaining form!</h2>
 <div className="form__section-left">
   <label>
     Full name
     <input
       type="text"
       name="name"
     />
   </label>
     // Rest of form inputs...
   </div>
 <input type="submit" value="Submit!" />
</form>;
```

We **add** an **event listener** to the **form** that takes a handler function

Uncontrolled forms

```
// JS
function handleSubmit(event) {
 event.preventDefault();

 const name = event.target.value;
 // Do stuff to submit data
}
```

And then the handler function will **receive** the form **event**, with its inputs, so you can then **submit** it using JS

Uncontrolled forms

```
// JS
function handleSubmit(event) {
 event.preventDefault();

 const name = event.target.value;
 // Do stuff to submit data
}
```

In React lingo, this **approach** is called “using an **Uncontrolled Component**”

You can read more about it in the React docs

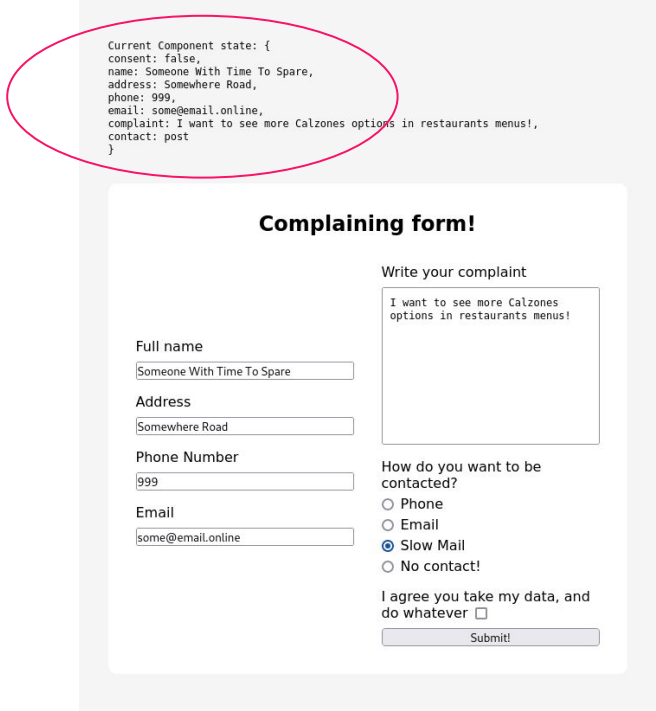
Q. Why do you think it’s called an Uncontrolled Component?

Controlled forms

A **Controlled form** has all of its data held by the **state** in a Component

This way we make sure all the state **changes** are **managed** by React. It makes it easier to write code to validate what the user is entering in the form (ie character limits) and to control, as a result, whether the submit is enabled or not.

Q. How can we connect the form with state?



Controlled forms

We first record any changes to the inputs in our state

We use the **onChange** event to listen for **new** user **inputs**, and we pass them to our **handleNameInput** handler function.

```
<form className="form" onSubmit={handleSubmit}>
 <h2>Complaining form!</h2>
 <div className="form__section-left">
   <label>
     Full name
     <input
       onChange={handleNameInput}
       type="text"
       name="name"
     />
   </label>
   // Rest of form inputs...
 </div>
 <input type="submit" value="Submit!" />
</form>;
```

Controlled forms

In its turn, **handleNameInput** will **update** the current **state** with the new value just received from the **event**

Q. What do you think it's happening in **setName**?

```
//JS
const [name, setName] = useState('');

const handleNameInput = event => {
 const inputValue = event.target.value;

 setName(inputValue);
};
```

Controlled forms

After we're able to update state with the new input value, we just need to make sure the form **input reflects** this updated state **value**

To do this, we need to set the **value** attr with the corresponding **state property**

```
<form className="form" onSubmit={handleSubmit}>
 <h2>Complaining form!</h2>
 <div className="form__section-left">
   <label>
     Full name
     <input
       onChange={updateFormState}
       type="text"
       name="name"
       value={name}
     />
   </label>
   // Rest of form inputs...
 </div>
 <input type="submit" value="Submit!" />
</form>;
```

Time for some live coding

Let’s build a simple controlled form!

Start here and then check the complete version here

Discuss potential ways to refactor the code and check the refactored version here

Form State Object

State per-field

```
const [firstName, setFirstName] = useState("");
const [lastName, setLastName] = useState("");
const [gender, setGender] = useState("");
const [terms, setTerms] = useState(false);
```

Form State Object

```
const [userData, setUserData] = useState({
  firstName: "",
  lastName: "",
  gender: "",
  terms: false
});
```

When we have many form fields, it can sometimes be simpler to use a single form state object rather than a state property per-field.

Form State Object

State per-field

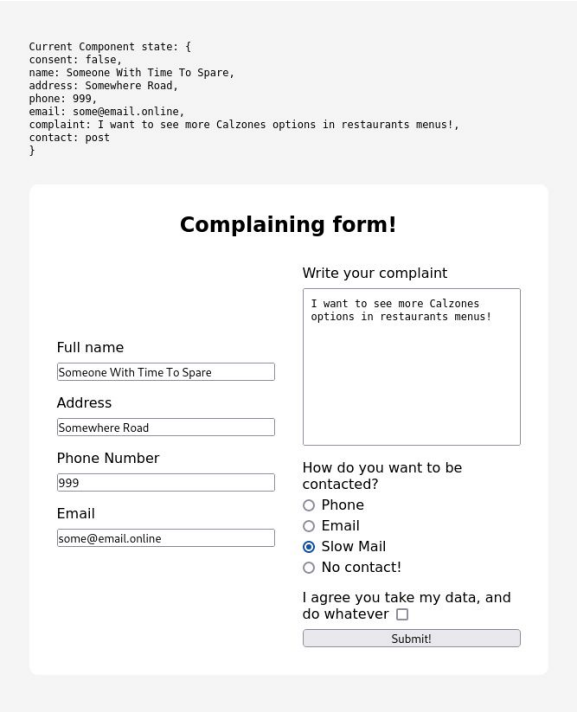
```
function handleFirstNameInput(event) {
   const inputValue = event.target.value;
   setFirstName(inputValue);
 }
 function handleLastNameInput(event) {
   const inputValue = event.target.value;
   setLastName(inputValue);
 }

 //etc...
```

Form State Object

```
function handleChange(event) {
  const inputName = event.target.name;
  const inputValue = event.target.value;
  const inputType = event.target.type;

  if (inputName === "firstName") {
    setUserData({ ...userData, firstName: inputValue });
  }
  if (inputName === "lastName") {
    setUserData({ ...userData, lastName: inputValue });
  }
  //etc...
}
```

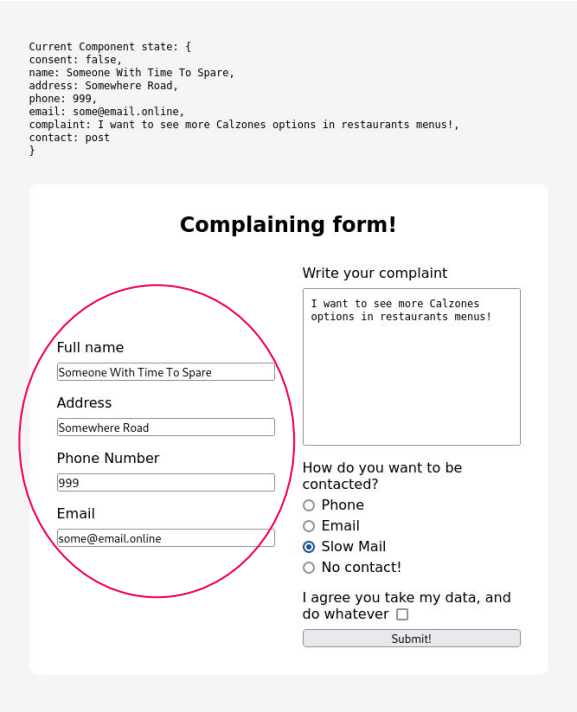
When the form is updated, we then update the state object with any new values using the spread syntax.

Controlled forms



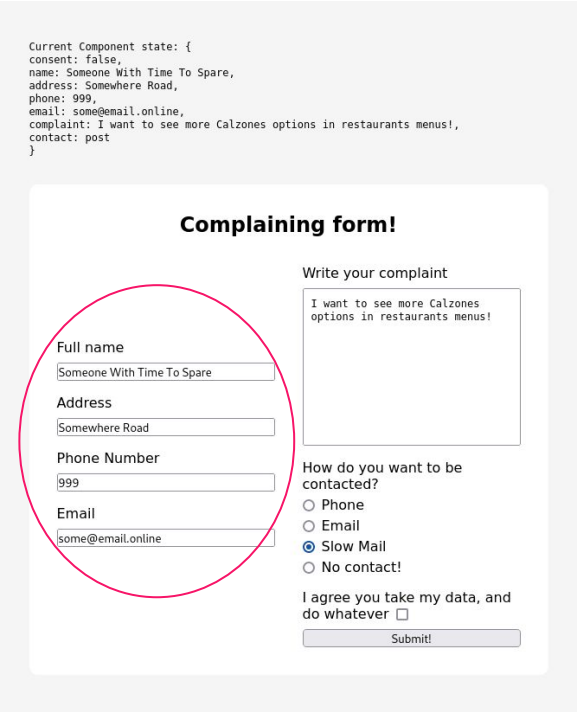
Now let's have a look at some different types of inputs, and how to use them with React

Controlled forms



We've seen the **text** input in action, and its cousins such as **email**, **password**, **number**, etc, which also interact with React in the **same** way.

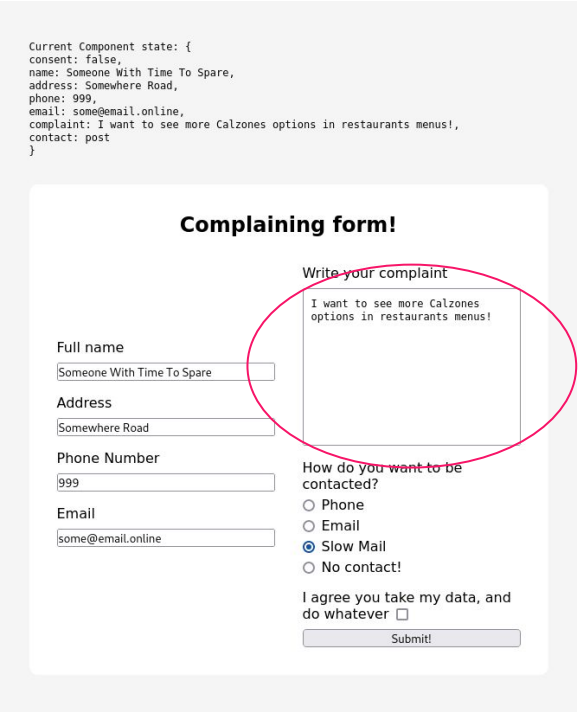
Controlled forms



These **inputs** are still just **HTML**, as such we can perform actions such as **validations**, by giving them the appropriate attributes

Q. How could I perform extra validations to an input value?

Controlled forms



Another similar input is the **textarea**.

Q. How could we display the current state value of a textarea?

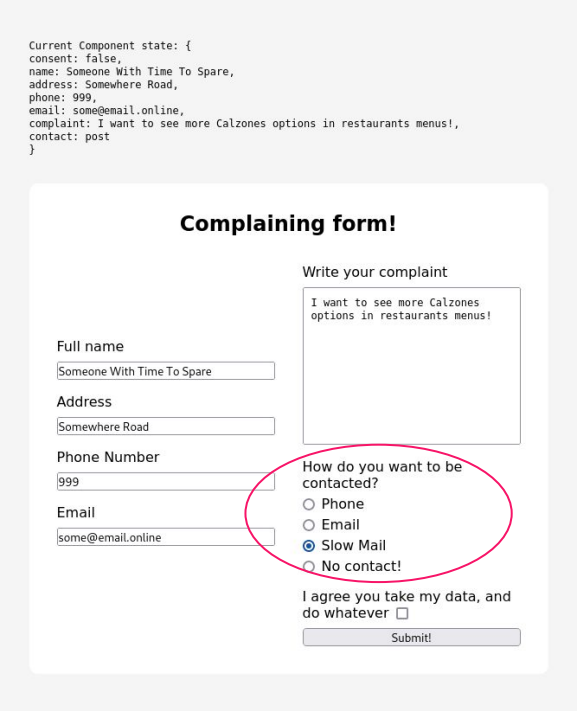
Controlled forms

```
<label>
 Write your complaint
 <textarea
   name="complaint"
   rows="10"
   placeholder="You can complain here"
   onChange={handleChange}
   value={form.complaint}
 >
 </textarea>
</label>
```

We can still put the **current** state **value** in the textarea's value attribute!

We want to give the **control** to **React**, so we shouldn't explicitly set any children to inputs, even though we could do it in this case!

Controlled forms



Here we have a radio buttons group! With this type of input, we only want to have **one** selected **at any given time**.

As such, we will have to store the **value** of the currently **selected** radio button in state

Controlled forms

```
<div className="form__radio-group">
 <p>How do you want to be contacted? </p>
 <label>
   <input
     onChange={handleChange}
     type="radio"
     name="contact"
     value="phone"
     checked={form.contact === "phone"}
   />
   Phone
 </label>
 // Rest of radio inputs
</div>;
```

Radio inputs get aggregated in a group by the **name** attribute

As such, all radio inputs should have the **same** name, as you will also use this attribute to **identify** the value in **state**

Controlled forms

```
<div className="form__radio-group">
 <p>How do you want to be contacted? </p>
 <label>
   <input
     onChange={handleChange}
     type="radio"
     name="contact"
     value="phone"
     checked={form.contact === "phone"}
   />
   Phone
 </label>
 // Rest of radio inputs
</div>
```

Every time a specific **radio** input within the group gets **selected**, the **onChange** listener will pass the **value** attribute to our handler function.

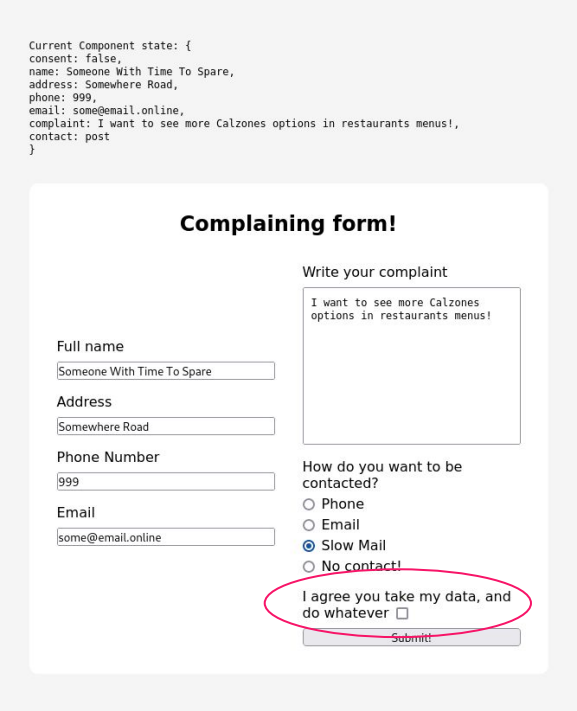
Controlled forms

```
<div className="form__radio-group">
 <p>How do you want to be contacted? </p>
 <label>
   <input
     onChange={handleChange}
     type="radio"
     name="contact"
     value="phone"
     checked={form.contact === "phone"}
   />
   Phone
 </label>
 // Rest of radio inputs
</div>;
```

To **display** which radio input is **selected**, we use the **checked** attribute

This attribute takes a **boolean**, so we need to **compare** the current **state** with the radio input **value** to find out if it's currently selected!

Controlled forms



Checkboxes work in a similar way to radio inputs.

The main difference is that, rather than tracking which one was checked, we want to keep track of **all** the checkboxes and their **checked value**.

Q. How would state look like in this case, compared with a radio input?

Controlled forms

```
<label>
 I agree you take my data, and do whatever
 <input
   onChange={handleChange}
   type="checkbox"
   name="consent"
   id="consent"
   checked={form.consent}
 />
</label>
```

Checkboxes also use the **checked** attribute to display if they've been checked

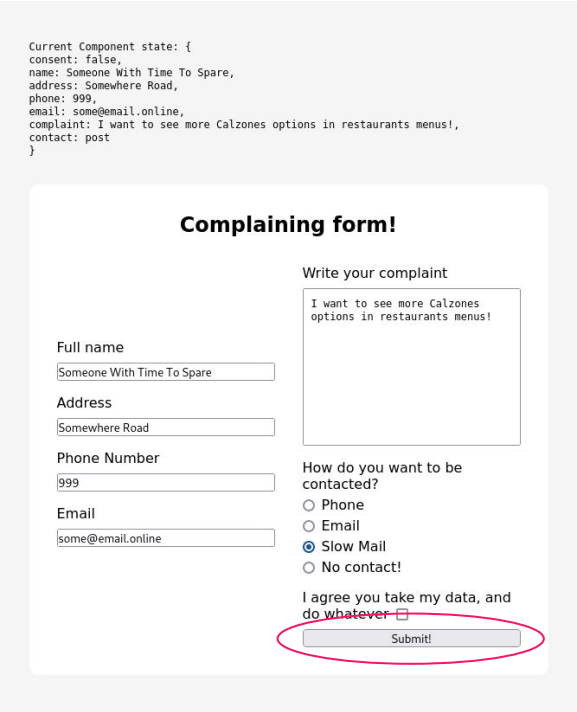
Controlled forms

```
const handleChange = (event) => {
 const { name, value, checked, type } = event.target;

 if (type === "checkbox" && name === "consent"){
  setForm({ ...form, consent: checked });
 }
 ...
};
```

To set it's value in state, we need to check for the **checked attribute**.

Then, we just use it as **value** to set the state, when the input **type** matches a **checkbox**!

Controlled forms



Submit inputs also take part of controlled forms in React!

Q. What do you think it should happen when we submit a controlled form?

Controlled forms

```
<form className="form" onSubmit={submitForm}>
 <h2>Complaining form!</h2>

  // Rest of form

 <input type="submit" value="Submit!" />
</form>;
```

The **submit** input will still trigger a **listener** that is set up in the form.

Controlled forms

```
const [form, setForm] = useState(initialFormState);

const submitForm = event => {
 event.preventDefault();

 fetch(URL, {
   method: "POST",
   // Rest of fetch
   body: JSON.stringify(form),
 });

 setForm(initialFormState);
};
```

In this instance, we use the submit handler to send the **current** form **state** to the server!

Controlled forms

```
const [form, setForm] = useState(initialFormState);

const submitForm = event => {
 event.preventDefault();

 fetch(URL, {
   method: "POST",
   // Rest of fetch
   body: JSON.stringify(form),
 });

 setForm(initialFormState);
};
```

For this, we use **fetch**, and give the **form** state as the **body**.

Controlled forms

```
const [form, setForm] = useState(initialFormState);

const submitForm = event => {
 event.preventDefault();

 fetch(URL, {
   method: "POST",
   // Rest of fetch
   body: JSON.stringify(form),
 });

 setForm(initialFormState);
};
```

And **after** we **submit** the form, we **set** the state back to its **initial** empty version!

Time for some practice

Let's give it a go!

https://github.com/boolean-uk/react-forms-practice